LOOPGUARD: INFINITE-LOOP DETECTION WITHIN THE LIMITS OF THE HALTING PROBLEM

CLA-2 Assessment Task 4: FLA-based real-world problem solving and system design (team work)

A COURSE ASSIGNMENT REPORT

submitted for the course

21CSC301T – FORMAL LANGUAGE AND AUTOMATA

Submitted by

RISHIKA VAMSHARAJ (RA2411026050069)

SREELAKSHMI A (RA2411026050058)

Under the guidance of

Dr.C.GUNASUNDARI

Assisstant Professor,

Dept of AIML

III Year / V Semester / Section A

B.Tech. Computer Science and Engineering with Specialization in Artificial Intelligence and Machine Learning

SCHOOL OF COMPUTING

FACULTY OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

TIRUCHIRAPPALLI – 621105

NOVEMBER 2026

CERTIFICATE

This is to certify that the assignment report titled *LoopGuard: Infinite-Loop Detection within the Limits of the Halting Problem*, submitted by **Rishika Vamsharaj (RA2411026050069)** and **Sreelakshmi A (RA2411026050058)** to the School of Computing, SRM Institute of Science and Technology, Tiruchirappalli, as the team work for Assessment Task 4 of Continuous Learning Assessment – 2 of the course **21CSC301T – Formal Language and Automata**, is a bona fide record of the work they carried out under the guidance of **DR.C.Gunasundari**, Assisstant Professor, Dept of AIML .

To the best of our knowledge, the contents of this report have not been submitted to any other university or institute for the award of any degree or diploma.

Dr.C.Gunasundari

Assisstant Professor,

Dept of AIML

Course Faculty

Head of the Department

Place: Tiruchirappalli

Date: November 2026

DECLARATION

We declare that this report is the team work of the two members named below for Assessment Task 4 of the Formal Language and Automata course. The results, tables and figures in it come from running the accompanying code, and the captured output logs are included in the submission package. Each member's contribution is stated in Chapter 9. Wherever we have used material from textbooks or papers, we have cited it. This report has not been copied from any other source.

Rishika Vamsharaj (RA2411026050069)

Sreelakshmi A (RA2411026050058)

Date: November 2026

ACKNOWLEDGEMENT

We thank Dr.C.Gunasundari, Assistant Professor, Department of AIML for guiding us through Formal Language and Automata and for setting a team task that links Turing machines and decidability to a problem every programmer meets. We are grateful to the School of Computing, SRM Institute of Science and Technology, Tiruchirappalli, for the facilities and support provided during the semester, and to our families and classmates for their encouragement.

ABSTRACT

Every programmer has watched a program freeze and wondered whether it is stuck or merely slow. This report treats that question, infinite-loop detection, as a problem about Turing machines. We state it as membership in the language LOOP, the complement of $HALT_{TM} = \{ \langle M, w \rangle : M \text{ halts on } w \}$. We prove with the diagonal argument that no algorithm decides it for every program, show that HALT is recognisable while LOOP is not even recognisable, and use Rice's theorem to place the related questions ‘always terminates’ and ‘program equivalence’ outside both classes. To make the proof concrete we execute the diagonal construction against five real candidate deciders, including a 300-step timeout rule, and each one is wrong on its own diagonal program.

We then design LoopGuard, a restricted-case detector for a small imperative language compiled to a register machine. It answers HALTS, LOOPS or UNKNOWN, and every definite answer carries its justification: a linear ranking function, a recurrent set, a constant-step argument, or a repeated machine configuration. A separate k-bit model decides the same programs exactly for fixed-width hardware. On an 18-program benchmark whose behaviour we derived by hand, LoopGuard gave 15 correct verdicts, 3 honest UNKNOWNs and 0 wrong answers. Over 4 537 generated programs no definite verdict disagreed with an independent oracle, and a bug-planting check showed that the sweep does detect faulty rules. A timeout-only watchdog, by contrast, called a correct but slow program an infinite loop. The complete code, tests and run logs are supplied with this report.

Keywords: Turing machine, halting problem, decidability, recognisability, Rice's theorem, termination analysis, ranking function, cycle detection, infinite-loop detection.

TABLE OF CONTENTS

Chapter 1 Introduction 10
1.1 Team and roles 10
1.2 Background 10
1.3 Organisation of the report 11

Chapter 2 Problem Definition and Formal Model 12
2.1 The computational problem 12
2.2 Why the model is faithful 13
2.3 Three questions of different difficulty 13

Chapter 3 Decidability Analysis 14
3.1 Is a general exact solution decidable? 14
3.2 Decidable versus recognisable 14
3.3 The argument, made executable 15
3.4 Three outcomes on Turing machines 16

Chapter 4 Restricted-Case Solution and System Design 17
4.1 Design goals 17
4.2 The program model: LoopLang and its register machine 17
4.3 System design 17
4.4 Stage 1: static proofs for affine loops 18
4.5 Stage 2: execution with configuration memory 19
4.6 Stage 3: the k-bit finite-state model 19
4.7 Soundness arguments 20

Chapter 5 Worked Examples 21
5.1 A loop that hides a real bug: Euclid with a zero 21
5.2 Why the machine model matters 21
5.3 An honest UNKNOWN 22

Chapter 6 Implementation 24
6.1 Key implementation decisions 24
6.2 Cost 24

Chapter 7 Validation and Results 25
7.1 Benchmark with hand-derived ground truth 25
7.2 Soundness sweeps 27

7.3 Does the sweep actually catch bugs? 27
7.4 Test suite 27
7.5 Threats to validity 28

Chapter 8 Practical Applications and Limitations 29
8.1 Practical implications 29
8.2 Limitations 29
8.3 Future work 29

Chapter 9 Individual Contribution Statements 31

Chapter 10 Source Code and Reproducibility 32

Chapter 11 Conclusion 33

References 33

LIST OF FIGURES

Fig. 1. From the real-world question to its formal language, what the theory says about it, and the engineering response

Fig. 2. The diagonal construction as a flowchart: any total decider H is refuted by the machine D built from it

Fig. 3. Where the loop-detection problems sit among the language classes (decidable = recognisable and co-recognisable)

Fig. 4. Output of our run of the executable diagonal argument against five candidate deciders, and of the Turing-machine simulator

Fig. 5. State diagrams of the three example Turing machines and their three possible outcomes

Fig. 6. LoopGuard system design: static proofs first, then execution with configuration memory, with the k-bit model reported alongside

Fig. 7. Flowchart of the static stage for one while loop (unbounded integers)

Fig. 8. LoopGuard explaining gcd_zero_bug: compiled code, each stage's answer and the final verdict

Fig. 9. wrong_direction: proved to loop on true integers, but it halts on an 8-bit machine

Fig. 10. flag_never_set: an honest UNKNOWN on true integers, yet exactly decidable (HALTS) on 8-bit hardware

Fig. 11. LoopGuard on the 18-program benchmark, with a timeout-only watchdog and the 8-bit model for comparison

Fig. 12. Benchmark outcome by detector (the watchdog's 17 'correct' include 10 unproved guesses of 'loops?')

Fig. 13. The two soundness sweeps and the full test-suite run

LIST OF TABLES

Table 1. Team roles
Table 2. Mapping between the real-world problem and its computational formulation
Table 3. Five candidate loop deciders and the diagonal program built against each
Table 4. LoopGuard techniques, why each one is sound, and the restriction it relies on
Table 5. Source layout of the code package
Table 6. Benchmark results for every program (ground truth derived by hand)
Table 7. Benchmark summary by detector
Table 8. Soundness sweeps and the bug-planting check of the sweep

Chapter 1 Introduction

Github : https://github.com/Sriiihhh/FLA-TASK-4

Assessment Task 4 of CLA-2 is the team task of the Formal Language and Automata course. It asks a team to take one real-world problem, model it with Turing machines, and explain what the theory of decidability says about solving it. We chose **infinite-loop detection in software**. It is the most direct real-world form of the halting problem, and every programmer has watched a program freeze and wondered whether it was stuck or only slow. Turing machines are not used to build real systems, but they fix what real systems can and cannot achieve; this report applies that idea to a problem we all meet.

The work has two halves. The first is theory: we state the problem as a formal language, prove with the diagonal argument that no algorithm decides it for every program, place it among the language classes, and run the diagonal construction against five real candidate deciders. The second is engineering: since an exact general answer is impossible, we designed and implemented **LoopGuard**, a detector for a restricted class of programs. It answers HALTS, LOOPS or UNKNOWN, and attaches a justification to every definite answer. We validated it on an 18-program benchmark, on 4 537 generated programs, and with a bug-planting check of the sweep itself.

In short, LoopGuard gave the correct verdict on 15 of the 18 benchmark programs, answered UNKNOWN on the other 3, and made no wrong claim anywhere, whereas a timeout-only watchdog labelled a correct but slow program as an infinite loop (Chapter 7).

1.1 Team and roles

Table 1. Team roles

Role	Member	Responsibility
Theory and formal modelling	Rishika Vamsharaj RA2411026050069	formal model of the problem, undecidability and recognisability argument, Rice's theorem, executable diagonal argument, Turing-machine simulator and examples
Design, implementation and validation	Sreelakshmi A RA2411026050058	LoopLang and its compiler, static analyzer, cycle detection and k-bit model, benchmark with hand-derived ground truth, sweeps and test suite
Documentation	both members	diagrams, report, and presentation; the benchmark table and the final report were reviewed together

1.2 Background

A Turing machine has a finite control and an unbounded read/write tape. A language is *Turing-recognisable* if some machine accepts exactly its strings, possibly looping on the others. It is *decidable* if some machine accepts its strings and also halts with ‘reject’ on every other input, and it is *co-recognisable* if its complement is recognisable. A language is decidable exactly when it is both recognisable and co-recognisable [1, 2]. The halting language $\mathrm{HALT}_{\mathrm{TM}} = \{ \langle M, w \rangle : M \text{ halts on } w \}$ is recognisable but not decidable [3]. Rice's theorem [4] extends this to every non-trivial property of what a program computes. When memory is bounded, however, a program

has only finitely many configurations and behaves like a finite automaton, for which questions about running forever are decidable [10]. This contrast between unbounded and bounded memory is the thread that runs through the report.

1.3 Organisation of the report

Chapter 2 defines the problem and its formal model. Chapter 3 analyses decidability and executes the diagonal argument. Chapter 4 presents the design of LoopGuard and Chapter 5 works through examples. Chapter 6 describes the implementation, Chapter 7 gives the validation results, and Chapter 8 discusses applications and limitations. Chapter 9 contains each member's contribution statement, Chapter 10 explains how to reproduce the results, and Chapter 11 concludes.

Chapter 2 Problem Definition and Formal Model

2.1 The computational problem

Real-world problem. Given the source code of a program P and the input it will receive, decide, before or while running it, whether P will run forever. A good answer would catch bugs such as a loop counter that moves the wrong way, a variable that is never updated, or a retry loop whose exit condition cannot occur, before the program freezes a phone, a build server or a web request.

Assumptions. Programs are deterministic. They read their whole input at the start and perform no further input or output, so a program together with its input fixes exactly one computation. For the formal argument, integers are unbounded, as a Turing machine's tape is. Section 4.6 also looks at what changes when integers have a fixed width, as on real hardware, because the answer to the same question can change.

Formal model. Any program in a general-purpose language can be compiled into a Turing machine, and any Turing machine can be simulated by a program, so nothing is lost by asking the question about Turing machines. The decision problems are:

```
HALT  = { ⟨M, w⟩ : M is a TM and M halts on input w }
LOOP  = { ⟨M, w⟩ : M runs forever on w }   = complement of HALT
TOTAL = { ⟨M⟩    : M halts on every input }   ('this program can never hang')
```

'Will P hang on w?' is exactly the question $\langle M_P, w\rangle \in$ LOOP, and 'can P hang on some input?' is $\langle M_P\rangle \notin$ TOTAL. Table 2 and Fig. 1 set out the correspondence.

Table 2. Mapping between the real-world problem and its computational formulation

Real world	Formal model
program P in C, Python or Java	Turing machine M_P
the input P is given	tape contents w
one run of P	the sequence of instantaneous descriptions (IDs) of M_P on w
P is stuck in an infinite loop	M_P never enters a halting state
'will P hang on this input?'	is $\langle M_P, w\rangle \in$ LOOP ?
'can P hang on some input?'	is $\langle M_P\rangle \notin$ TOTAL ?
a machine with k-bit variables	a finite automaton with at most #pc · $2^{k\cdot v}$ configurations
a loop-detection tool	a TM that is meant to decide LOOP

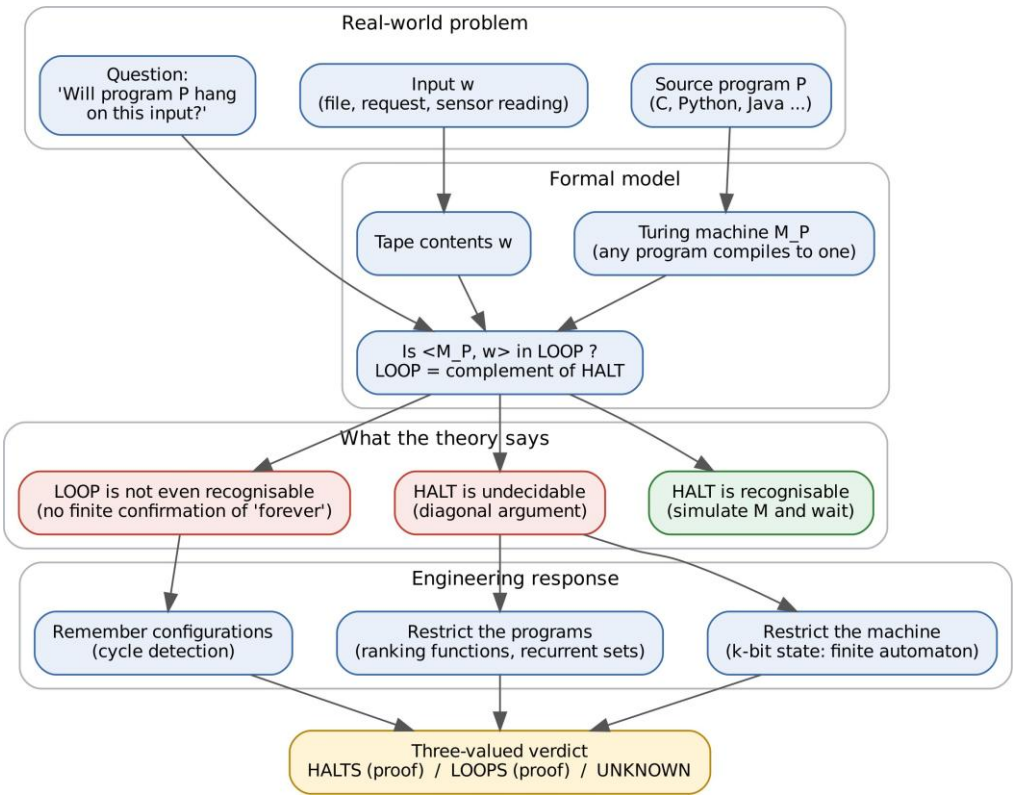


Fig. 1. From the real-world question to its formal language, what the theory says about it, and the engineering response

2.2 Why the model is faithful

A program's state at any moment is its program counter together with the values of its variables. This *configuration* plays the role of a Turing machine's instantaneous description, and one step of the program corresponds to one move of the machine. Because the program is deterministic, a configuration determines the entire future of the run. Two consequences drive the rest of the report. First, if the same configuration ever occurs twice, the run repeats forever, which is a finite proof of non-termination. Second, if the memory is unbounded and the configurations keep changing, no such finite evidence need ever appear. A register machine with unbounded integers can simulate a Turing machine and vice versa, so the undecidability results proved for Turing machines hold for the small language we use in Chapter 4.

2.3 Three questions of different difficulty

- **One program, one input** (⟨M, w⟩ ∈ LOOP): undecidable, and LOOP is not even recognisable.
- **One program, all inputs** (⟨M⟩ ∈ TOTAL): undecidable and neither recognisable nor co-recognisable, by Rice's theorem.
- **Bounded memory** (k-bit variables): decidable, because the machine is a finite automaton, although the number of configurations grows exponentially with k.

Chapter 3 Decidability Analysis

3.1 Is a general exact solution decidable?

No. We give the standard diagonal proof [1, 3], written for our problem and shown as a flowchart in Fig. 2.

Suppose some program H always halts and correctly answers H(⟨M, w⟩) = ‘halts’ or ‘loops’. Build a new machine D that, on input ⟨M⟩, runs H on ⟨M, ⟨M⟩⟩. If H says ‘halts’, D enters an infinite loop. If H says ‘loops’, D halts. Now run D on its own description ⟨D⟩. If D(⟨D⟩) halts, then H(⟨D, ⟨D⟩⟩) must have said ‘loops’, which is wrong. If D(⟨D⟩) loops, then H said ‘halts’, which is also wrong. Either way H is wrong on at least one input, so no such H exists. HALT is undecidable, and so is LOOP, because decidable languages are closed under complement: a decider for either would give a decider for the other.

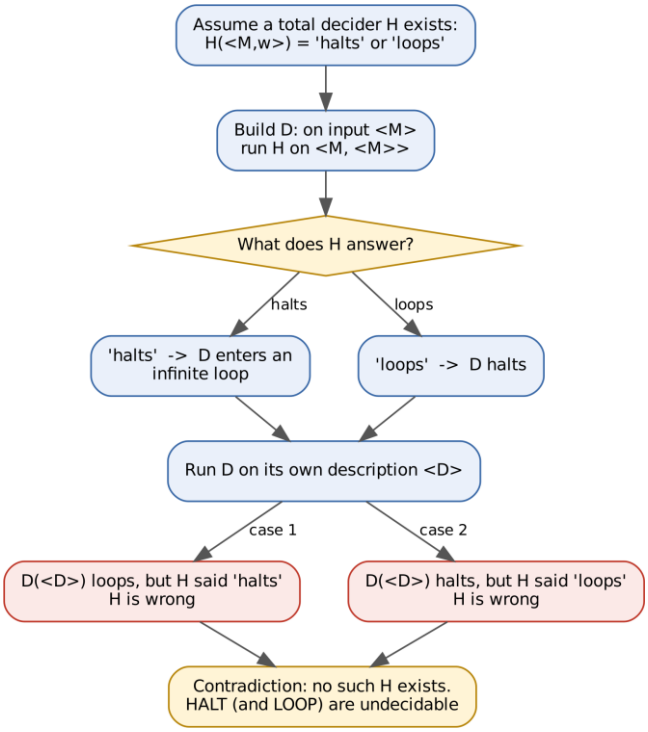


Fig. 2. The diagonal construction as a flowchart: any total decider H is refuted by the machine D built from it

3.2 Decidable versus recognisable

The two languages are not equally hard.

- **HALT is Turing-recognisable.** Simulate M on w and accept if it halts. If M halts we will see it after finitely many steps; if it does not, the simulation runs forever, which a recogniser is allowed to do.
- **LOOP is not even recognisable.** If it were, HALT and its complement would both be recognisable, and HALT would be decidable, which it is not. In plain terms, no procedure confirms every infinite loop after a finite amount of work.
- **TOTAL is neither recognisable nor co-recognisable** [1, 2]. More generally, by Rice's theorem [4], every non-trivial property of the function a program computes, including ‘always

terminates' and 'is equivalent to this other program', is undecidable.

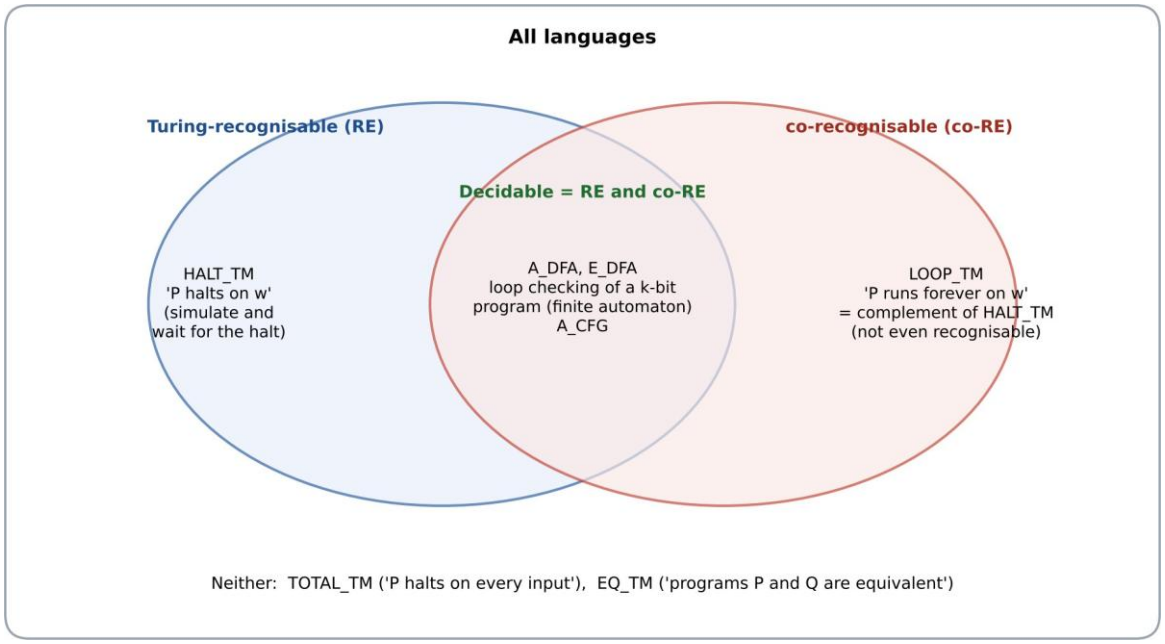


Fig. 3. Where the loop-detection problems sit among the language classes (decidable = recognisable and co-recognisable)

This asymmetry explains an everyday experience: we can always confirm that a program has finished, but we can never be sure that a slow program is stuck rather than just slow. It also explains why any practical tool must be allowed a third answer.

3.3 The argument, made executable

A proof on paper is convincing, but we also wanted to watch the construction defeat real candidates. In *loopguard/diagonal.py* a program is a Python generator in which every *yield* is one step, and a candidate decider H is itself a generator that returns 'halts' or 'loops', so that H can be run *inside* the program built against it, as the proof requires. The function make_diagonal(H) builds D exactly as in the proof. We tried five candidates (Table 3): one that always says 'halts', one that always says 'loops', one that searches the source text for `while True`, one that simulates the program for 300 steps and says 'loops' if it has not stopped, and a hybrid of the last two. Every candidate is a real, terminating function, and every one is wrong on its own D (Fig. 4).

Table 3. Five candidate loop deciders and the diagonal program built against each

Candidate	Rule	H(⟨D⟩)	D really	Verdict
H_optimist	says every program halts	halts	loops	H wrong
H_pessimist	says every program loops	loops	halts (step 1)	H wrong
H_syntactic	looks for 'while True' in the source	loops	halts (step 1)	H wrong
H_timeout	runs P for 300 steps; 'loops' if not stopped	loops	halts (step 301)	H wrong
H_hybrid	no 'while' -> halts, else the timeout rule	loops	halts (step 301)	H wrong

The simple deciders also fail on ordinary programs, without any self-reference. The timeout rule calls a program that merely needs 301 steps an infinite loop, and the syntactic rule calls a loop that exits through a `break` an infinite loop. Both failures are visible at the end of the first block of Fig. 4.

```
LoopGuard - diagonal argument and Turing machines

$ python3 -m loopguard.cli theory
================================================================================
1. Diagonal argument against five candidate deciders
================================================================================
   candidate H  what it does                                  H says D does
   H_optimist   says every program halts                      halts  loops   H WRONG
   H_pessimist  says every program loops                      loops  halts   H WRONG
   H_syntactic  looks for 'while True' in the source          loops  halts   H WRONG
   H_timeout    runs P for 300 steps; 'loops' if not stopped  loops  halts   H WRONG
   H_hybrid     no 'while' -> halts, else the timeout rule    loops  halts   H WRONG

   Ordinary programs that also defeat the simple deciders:
   H_timeout    on slow_but_halting  says loops  but it halts (after 302 steps)
   H_syntactic  on retry_with_break  says loops  but it halts (after 6 steps)

================================================================================
2. Three Turing machines, three outcomes
================================================================================
   binary incrementer on '1011': HALTS after 8 steps
      IDs: [right]1011 |- 1[right]011 |- 10[right]11 |- 101[right]1 |- 1011[right]_ |- ...
   ping-pong on '0': LOOPS after 2 steps
      IDs: [a]0 |- 0[b]_ |- [a]0_
      the ID at step 2 equals the ID at step 0: deterministic, so it repeats forever
   runaway writer on '': UNKNOWN after 2000 steps
      IDs: [w]_ |- 1[w]_ |- 11[w]_ |- 111[w]_ |- 1111[w]_ |- ...
      tape keeps growing, no ID repeats: no detector can say more
```

Fig. 4. Output of our run of the executable diagonal argument against five candidate deciders, and of the Turing-machine simulator

3.4 Three outcomes on Turing machines

The second block of Fig. 4 shows our Turing-machine simulator on three small machines (Fig. 5). A binary incrementer halts. A two-state ‘ping-pong’ machine repeats an instantaneous description after 2 steps, which *proves* that it loops: the machine is deterministic, so the same ID is followed by the same sequence forever. A machine that keeps writing 1s to the right never halts, but its ID never repeats because the tape keeps growing, so any detector can only report UNKNOWN once its budget runs out. These three outcomes, halts, proven loop and unknown, structure the design of LoopGuard.

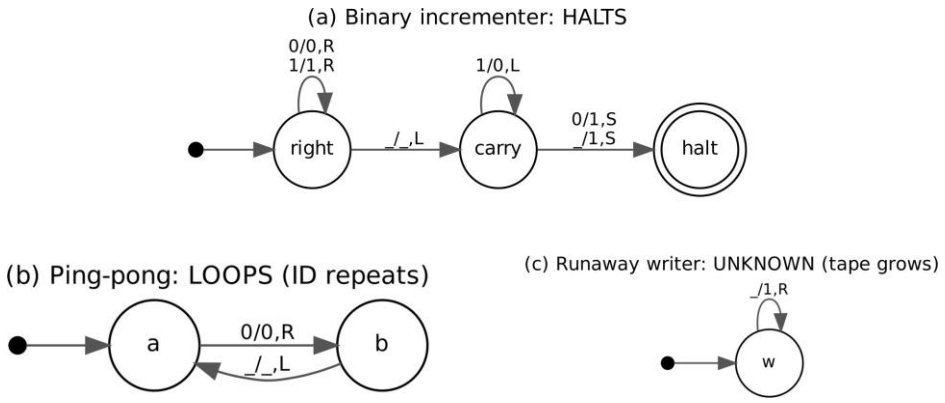


Fig. 5. State diagrams of the three example Turing machines and their three possible outcomes

Chapter 4 Restricted-Case Solution and System Design

4.1 Design goals

Since no exact general algorithm exists, LoopGuard aims for what can exist: a detector that is **sound** on the cases it handles and says so when it cannot decide. It gives one of three verdicts: HALTS, LOOPS, or UNKNOWN. Every HALTS and LOOPS carries a justification that a reader can check, and a wrong HALTS or LOOPS would be a bug. UNKNOWN is the undecidability result showing up in practice, and we treat it as a correct output, never as a failure to be hidden by guessing.

4.2 The program model: LoopLang and its register machine

Programs are written in LoopLang, which uses Python syntax but allows only integer variables, assignment and augmented assignment, `while`, `if`/`else`, `break` and `pass`, with the operators + − × // % and comparisons, `and`/`or`/`not`. Variables start at 0 unless an input value is supplied. The Python *ast* module [14] parses the source and a validator rejects everything else. Each program is compiled into register-machine code with four instructions: *set*, *jf* (jump if false), *jmp* and *halt*. For example, `n = 10; while n > 0: n -= 1` becomes:

```
0: n := 10
1: if not (n > 0) goto 4
2: n := n - 1
3: goto 1
4: halt
```

The configuration (program counter, values of all variables) plays the role of the Turing machine's instantaneous description. Register machines with unbounded integers are as powerful as Turing machines, so the undecidability result applies to LoopLang in full; the restriction we exploit is in what the *analyzer* understands, not in what the language can express.

4.3 System design

Fig. 6 shows the pipeline. The source and the input values are parsed and compiled. Stage 1 tries to *prove* the verdict without running anything. If it finds no proof, Stage 2 runs the machine while remembering configurations. Independently, Stage 3 evaluates the same program on a k-bit machine, where the question becomes decidable, and reports that answer next to the main verdict.

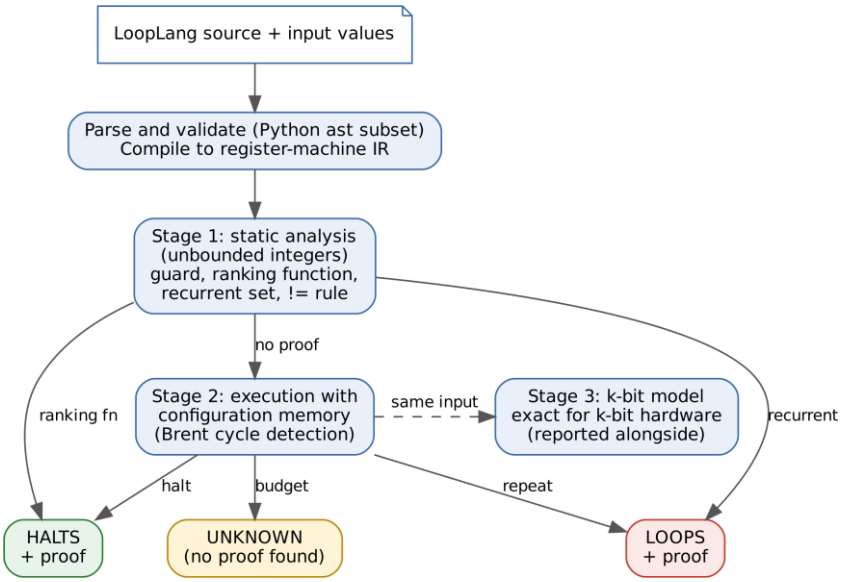


Fig. 6. LoopGuard system design: static proofs first, then execution with configuration memory, with the k-bit model reported alongside

4.4 Stage 1: static proofs for affine loops

Stage 1 walks the top-level statements, executing straight-line assignments with the known values so that each loop is analysed with its concrete entry values. For a loop `while G: body` whose guard G compares two linear expressions, the guard is normalised to the form $t \geq 0$, where $t = \sigma \cdot (\text{lhs} - \text{rhs}) - m$ with $\sigma = \pm 1$ and $m \in \{0, 1\}$. If every guard variable changes only by constants, once per pass, at the top level of the body, then t changes by a constant d per pass. With t_0 the value on entry, the rules of Table 4 apply, and Fig. 7 shows the decision flow.

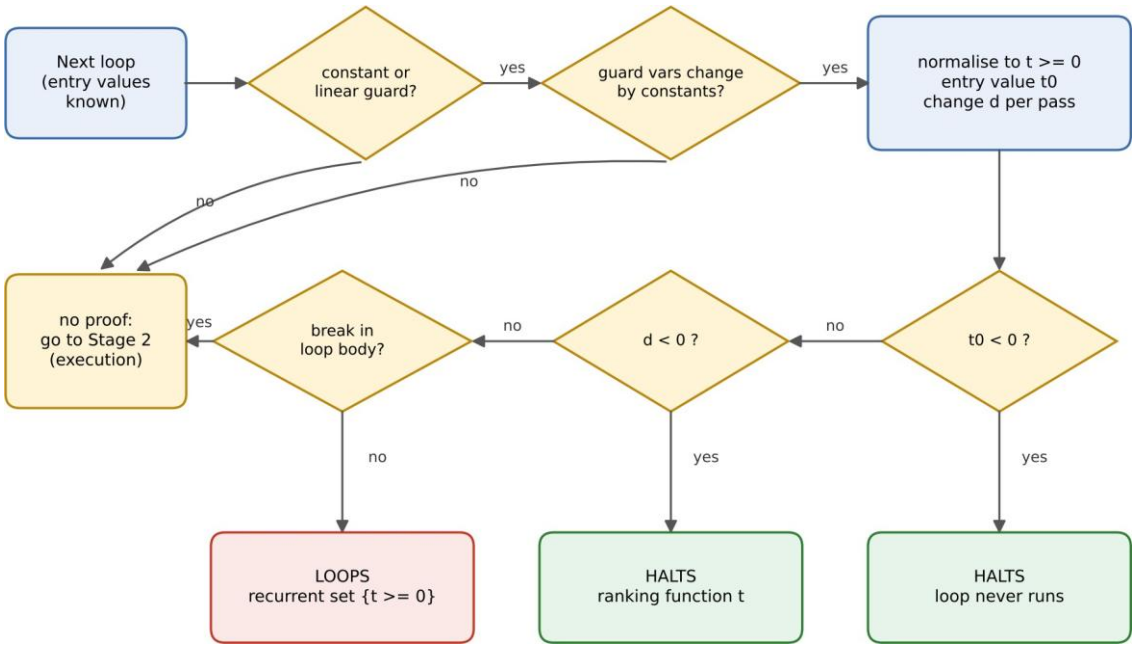


Fig. 7. Flowchart of the static stage for one while loop (unbounded integers)

Table 4. LoopGuard techniques, why each one is sound, and the restriction it relies on

Technique	Proves	Why it is sound	Restriction that makes it decidable
Constant guard	HALTS or LOOPS	`while False` never runs; `while True` with no break never exits	the guard is a literal
Linear ranking function [6, 9]	HALTS	t is an integer, $t \geq 0$ while the loop runs, and t falls by at least 1 per pass, so there are at most $\lfloor t_0/\lvert d\rvert \rfloor + 1$ passes	linear guard; guard variables change only by constants; no nested loop
Recurrent set [7]	LOOPS	$t_0 \geq 0$ and $d \geq 0$ give $t \geq 0$ after every pass, so the guard never becomes false	same restriction, and no break in the loop
!= and == rules	HALTS or LOOPS	s = lhs − rhs equals $s_0 + k \cdot d$ after k passes; it reaches 0 exactly when $d \neq 0$, d divides s_0 and the quotient is positive	one constant step per pass
Configuration memory [8]	HALTS (seen) or LOOPS (repeat)	the machine is deterministic, so a repeated configuration repeats its future forever	finds only loops that revisit a configuration
k-bit finite model [10]	exact HALTS or LOOPS	at most $\#pc \cdot 2^{kv}$ configurations, so a run must halt or repeat (pigeonhole)	values wrap modulo 2^k; the program becomes a finite automaton

Loops that do not fit these templates, for example `x = x * 2` or loops whose guard variables change inside an `if`, receive no static answer and go to Stage 2. A loop proved to halt is then executed concretely, which is safe because the proof bounds the number of passes, so that later statements can be analysed with the correct values.

4.5 Stage 2: execution with configuration memory

Stage 2 interprets the register code for at most a budget of 100 000 steps. If the halt instruction is reached, the verdict is HALTS, justified by the run itself. If a configuration recurs, the verdict is LOOPS: the machine is deterministic, so it will cycle forever. Recurrence is found with Brent's cycle-detection algorithm [8], which saves a single configuration at steps 1, 3, 7, 15, … and compares each new configuration with it. This needs memory for one configuration only, and it reports a repeat after at most about $3(\mu + \lambda)$ steps, where μ is the number of steps before the cycle starts and λ the cycle length. If the budget is spent with no halt and no repeat, the verdict is UNKNOWN. This is the case when values keep growing or a counter keeps changing, because then no configuration ever repeats.

4.6 Stage 3: the k-bit finite-state model

If every variable is a k-bit register, the program has at most $\#pc \cdot 2^{k \cdot v}$ configurations, where v is the number of variables, so it is a finite automaton, and for finite automata ‘does it run forever?’ is decidable. LoopGuard therefore re-runs the program with signed k-bit wrap-around arithmetic (k = 8 by default) and, by the pigeonhole principle, gets an exact HALTS or LOOPS within a budget of $4 \cdot \#pc \cdot 2^{kv} + 16$ steps. This ties the task back to finite automata: undecidability needs unbounded memory. The catch is size, since 8 bits and 3 variables already give 2^{24} configurations per program-counter value, so the implementation caps the work at 4 000 000 steps and answers UNKNOWN beyond it. Input values and constants are reduced modulo 2^k when this model is used.

4.7 Soundness arguments

- **Ranking function.** At each loop head the guard holds only if $t \geq 0$. After n passes $t = t_0 - n|d|$, so the guard can hold at the head of pass n+1 only if $n \leq t_0/|d|$. Hence at most $\lfloor t_0/|d| \rfloor + 1$ passes occur. Division by zero inside the body would end the program abnormally, which is also termination.
- **Recurrent set.** If $t \geq 0$ at some head and $d \geq 0$, then $t + d \geq 0$ at the next head, so by induction the guard is true forever. A break would invalidate this, so loops with a break are excluded. A nested loop cannot help the program to exit, because any exit would have to happen at this loop's head.
- **Configuration repeat and k-bit model.** Determinism gives $C_i = C_j \Rightarrow C_{i+k} = C_{j+k}$ for all k, so the run is periodic. Brent's algorithm reports a repeat only when the compared configurations are equal, so a LOOPS verdict is never a guess.

Chapter 5 Worked Examples

5.1 A loop that hides a real bug: Euclid with a zero

Take the subtraction form of Euclid's algorithm with inputs a = 0 and b = 5:

```
while a != b:
    if a > b:
        a -= b
    else:
        b -= a
```

Stage 1 gives up, and rightly so: the guard variables a and b change inside an `if`, by different amounts on the two branches, so no ranking template applies. Stage 2 then runs the machine while remembering configurations (Fig. 8). Because a = 0, the else-branch does b −= 0, the configuration comes back unchanged after 4 steps, and the repeat proves the loop. The same code with a = 1071 and b = 462 halts, as the benchmark confirms in Chapter 7. The bug is real: Euclid by subtraction does not handle a zero argument.

```
LoopGuard - explain gcd_zero_bug

$ python3 -m loopguard.cli explain gcd_zero_bug
================================================================================
Program 'gcd_zero_bug'  inputs={'a': 0, 'b': 5}  hand-checked truth=loops
================================================================================
   while a != b:
       if a > b:
           a -= b
       else:
           b -= a

compiled register-machine code
    0: if not (a != b) goto 6
    1: if not (a > b) goto 4
    2: a := a - b
    3: goto 5
    4: b := b - a
    5: goto 0
    6: halt

stage 1 static analysis : UNKNOWN - no ranking function, recurrent set or constant-step
                          argument applies
stage 2 execution       : LOOPS - configuration at step 7 equals the one at step 3 (cycle
                          length 4)
stage 3 8-bit machine   : LOOPS - configuration at step 7 equals the one at step 3 (cycle
                          length 4)
VERDICT (unbounded ints): LOOPS  [execution]
```

Fig. 8. LoopGuard explaining gcd_zero_bug: compiled code, each stage's answer and the final verdict

5.2 Why the machine model matters

The loop `i = 10; while i > 0: i += 1` really loops on mathematical integers, and the recurrent-set rule proves it statically with $t_0 = 9$ and $d = +1$. On an 8-bit machine, however, i overflows from 127 to −128 and the loop halts after 356 steps. LoopGuard reports both answers side by side (Fig. 9). The same source text can terminate or not depending on the machine model, which is why stating assumptions, as the task requires, actually matters.

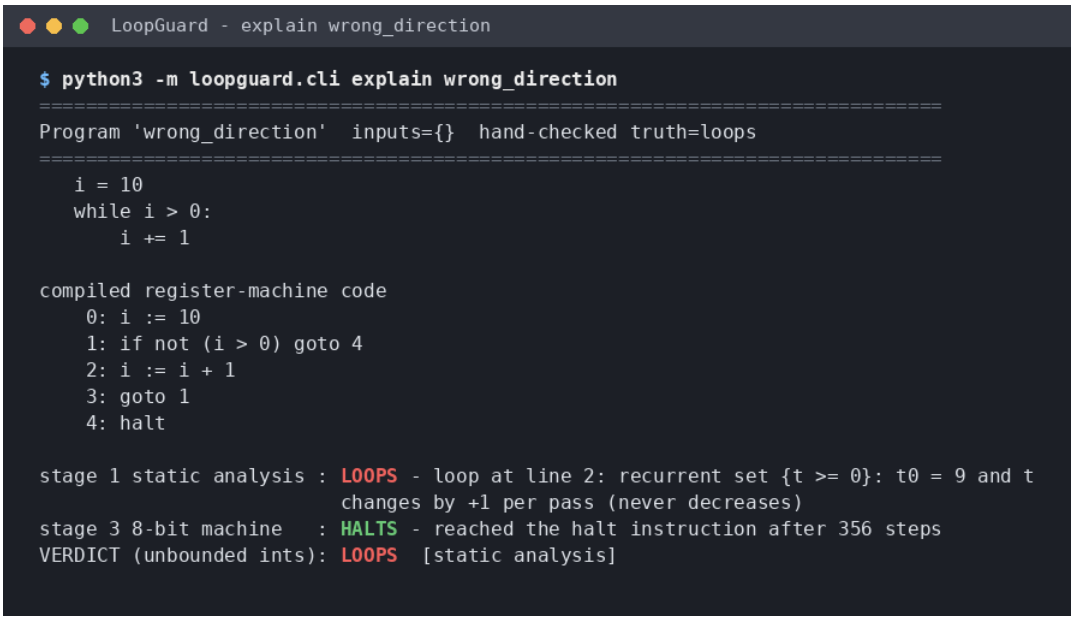


Fig. 9. wrong_direction: proved to loop on true integers, but it halts on an 8-bit machine

5.3 An honest UNKNOWN

The program in Fig. 10 models a ‘wait for a flag’ loop whose flag is set only if a counter becomes negative. On true integers the counter only grows, so the flag is never set and the program loops forever. Stage 1 finds no proof because the guard variable is assigned inside an `if`, and Stage 2 sees a counter that changes at every pass, so no configuration repeats within the budget. LoopGuard therefore answers UNKNOWN. This is the correct output, not a defect: any extra rule we add would leave another program outside it. Yet on the 8-bit machine the counter wraps from 127 to −128 and the loop exits after 516 steps, which shows the unbounded-versus-bounded contrast from Chapter 2 in a single program.

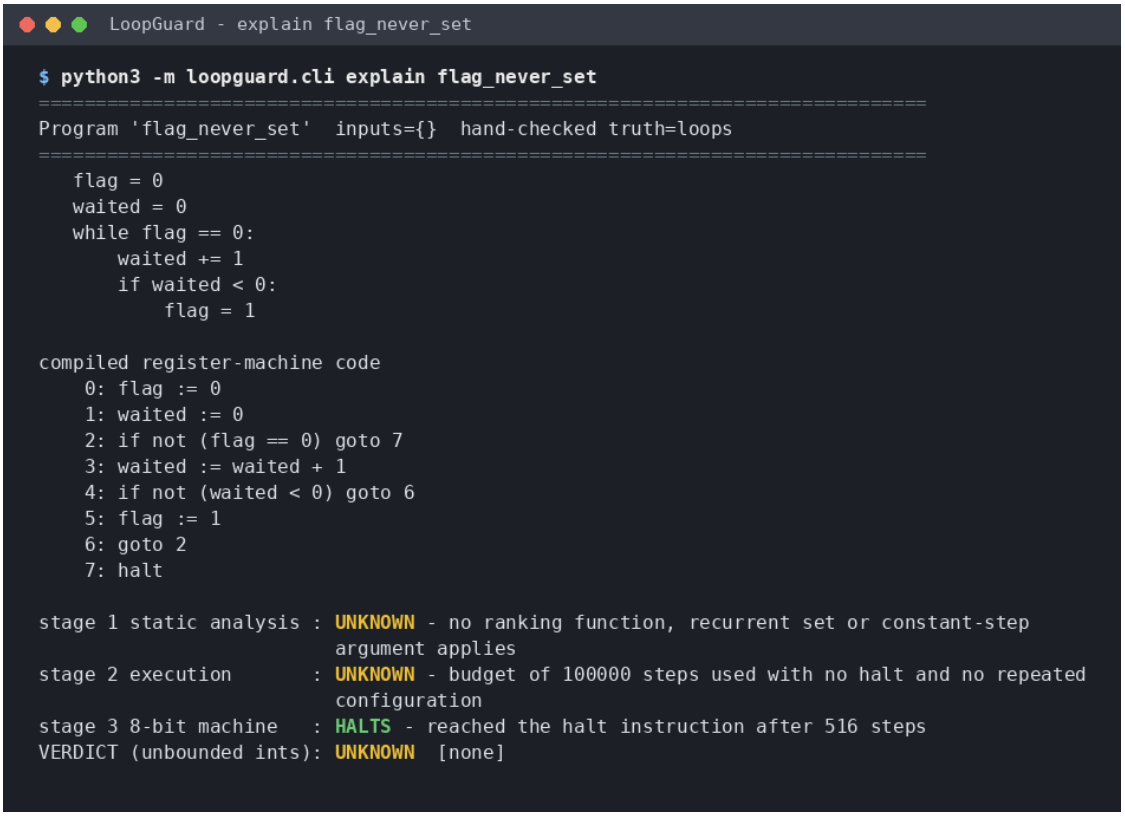


Fig. 10. flag_never_set: an honest UNKNOWN on true integers, yet exactly decidable (HALTS) on 8-bit hardware

Chapter 6 Implementation

LoopGuard is implemented in Python 3 with no third-party dependencies (pytest is needed only to run the tests, and Graphviz and Matplotlib only to redraw the figures). The package has 1068 lines in total, including 141 lines of tests. Table 5 lists the modules.

Table 5. Source layout of the code package

Path	Contents
loopguard/lang.py	LoopLang validator, compiler to register-machine IR, unbounded and k-bit evaluation
loopguard/static.py	Stage 1: guard normalisation, ranking-function, recurrent-set and != / == rules, concrete walk of proven loops
loopguard/dynamic.py	Stage 2: Brent cycle detection; exact k-bit decision; timeout-only watchdog baseline
loopguard/engine.py	the three-stage pipeline returning HALTS, LOOPS or UNKNOWN with the method used
loopguard/diagonal.py	executable diagonal argument against five candidate deciders
loopguard/tm.py	Turing-machine simulator with instantaneous-description repetition detection
loopguard/programs.py, bench.py	18-program benchmark with hand-derived ground truth, and its driver
loopguard/sweep.py, cli.py	two generated-program sweeps with independent oracles; command-line front end
tests/test_loopguard.py	39 tests
logs/, docs/	captured output of every run shown in this report; diagram sources and images

6.1 Key implementation decisions

- **One evaluator per model.** Unbounded arithmetic uses Python's integers through compiled expression objects; k-bit arithmetic uses a separate evaluator that wraps the result of every operation to a signed k-bit value.
- **Proof-carrying results.** Each static verdict returns the text of its proof (for example the entry value t_0 and the step d), and each execution verdict returns the two step numbers of the repeated configuration, so the user sees *why*.
- **Abnormal termination.** Division by zero is reported as termination, because the program stops; the soundness arguments rely on this.
- **Budgets.** The execution stage uses 100 000 steps; the k-bit stage is capped at 4 000 000 steps; a loop proved to halt is replayed concretely only if its pass bound is at most 2 000 000. Beyond a cap the answer is UNKNOWN, never a guess.

6.2 Cost

Stage 1 is linear in the size of the program plus the concrete replay of any proven loop. Stage 2 costs at most the budget in steps and keeps one saved configuration. Stage 3 costs at most $4 \cdot \#pc \cdot 2^{kv} + 16$ steps. The whole 18-program benchmark, including the k-bit model and the watchdog baseline, ran in 0.5 seconds on our machine.

Chapter 7 Validation and Results

7.1 Benchmark with hand-derived ground truth

We assembled a benchmark of 18 LoopLang programs (8 that halt and 10 that run forever on unbounded integers) and worked out the true behaviour of each one by hand, before running the tool; the reason for each answer is stored beside the program in *programs.py*. The set contains ordinary loops, a nested loop, common bugs (wrong direction, forgotten update, gcd with a zero, a step that skips the target), an intentional server loop, a modular cycle, a flag that is never set, and Collatz from 27 [13]. The test suite additionally re-runs every halting program for up to five million steps to confirm that it really halts. Table 6 gives the result for each program and Table 7 summarises it.

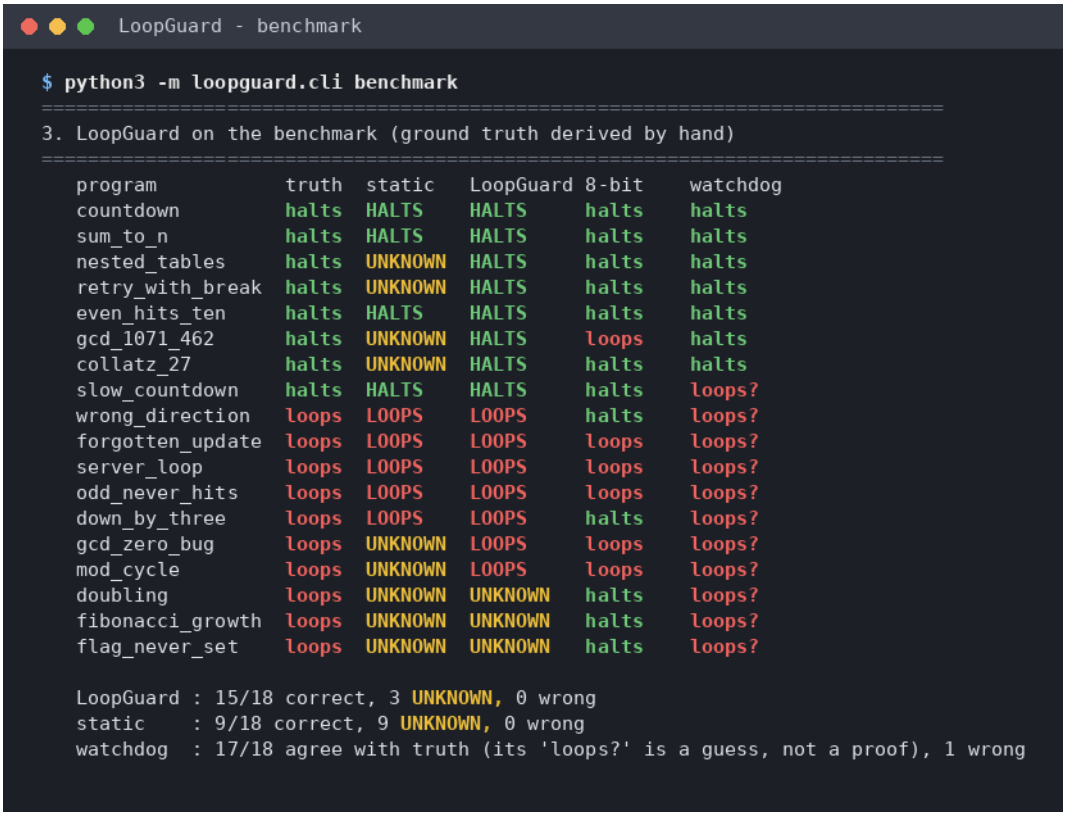


Fig. 11. LoopGuard on the 18-program benchmark, with a timeout-only watchdog and the 8-bit model for comparison

Table 6. Benchmark results for every program (ground truth derived by hand)

Program	Truth	Static stage	LoopGuard	8-bit model	Watchdog
`countdown`	halts	HALTS	HALTS (static)	halts	halts
`sum_to_n`	halts	HALTS	HALTS (static)	halts	halts
`nested_tables`	halts	UNKNOWN	HALTS (run)	halts	halts
`retry_with_break`	halts	UNKNOWN	HALTS (run)	halts	halts
`even_hits_ten`	halts	HALTS	HALTS (static)	halts	halts
`gcd_1071_462`	halts	UNKNOWN	HALTS (run)	loops	halts
`collatz_27`	halts	UNKNOWN	HALTS (run)	halts	halts
`slow_countdown`	halts	HALTS	HALTS (static)	halts	loops?
`wrong_direction`	loops	LOOPS	LOOPS (static)	halts	loops?
`forgotten_update`	loops	LOOPS	LOOPS (static)	loops	loops?
`server_loop`	loops	LOOPS	LOOPS (static)	loops	loops?
`odd_never_hits`	loops	LOOPS	LOOPS (static)	loops	loops?
`down_by_three`	loops	LOOPS	LOOPS (static)	halts	loops?
`gcd_zero_bug`	loops	UNKNOWN	LOOPS (run)	loops	loops?
`mod_cycle`	loops	UNKNOWN	LOOPS (run)	loops	loops?
`doubling`	loops	UNKNOWN	UNKNOWN	halts	loops?
`fibonacci_growth`	loops	UNKNOWN	UNKNOWN	halts	loops?
`flag_never_set`	loops	UNKNOWN	UNKNOWN	halts	loops?

Table 7. Benchmark summary by detector

Detector	Correct	UNKNOWN	Wrong	Notes
LoopGuard (static, then run with cycle detection)	15	3	**0**	every definite answer carries a justification
Static stage alone	9	9	0	fast; proves the textbook loops
Timeout-only watchdog (50 000 steps)	17*	–	1	*its ‘loops?’ answers are guesses; it calls `slow_countdown` an infinite loop

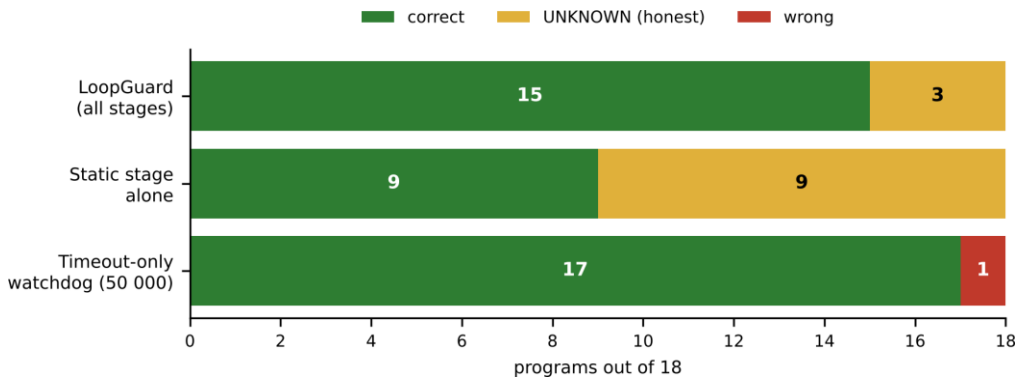


Fig. 12. Benchmark outcome by detector (the watchdog's 17 'correct' include 10 unproved guesses of 'loops?')

What the results show. (i) The static stage alone decides 9 programs and is never wrong, but returns UNKNOWN on 9. (ii) Execution with cycle detection adds 6 more verdicts (nested_tables, retry_with_break, gcd_1071_462, collatz_27, gcd_zero_bug, mod_cycle), so the full pipeline decides 15 of 18. (iii) The 3 UNKNOWN programs are `doubling`, `fibonacci_growth`, `flag_never_set`. All three really loop on unbounded integers, but their values or counters keep changing, so no configuration repeats and none fits a ranking template. We could add a rule for each one, but there would always be another program outside the new rules, which is exactly what Section 3.1 proves. (iv) The watchdog agrees with the truth on 17 programs, but 10 of those are unproved guesses of 'loops?', and it wrongly labels the correct, slow program `slow_countdown` (900 000 steps) as looping. (v) In the 8-bit model 6 of the 18 programs behave differently from the unbounded truth (gcd_1071_462, wrong_direction, down_by_three, doubling, fibonacci_growth, flag_never_set); for gcd_1071_462 the reason is that its inputs do not fit in 8 bits and are reduced modulo 256.

7.2 Soundness sweeps

To test soundness beyond 18 hand-picked cases, *sweep.py* generates programs and compares every definite verdict with an oracle that does not use LoopGuard code. **Sweep A** has 2 137 affine loops: a single variable with six comparison operators, seven step sizes, nine starting values and four bounds, plus a two-variable loop `while i < j: i += a; j -= b`. The oracle runs the loop natively for up to 10 000 passes; with these small parameters any terminating loop stops within about 100 passes, so a loop still running is a true infinite loop. **Sweep B** has 2 400 non-affine modular loops `x = (A*x + B) % M` with the guard `x != T`, which Stage 1 cannot analyse, so they exercise Stage 2. Here the oracle is exact: the state x lives in range(M), so after M + 2 steps a state must repeat. Table 8 lists the outcome.

Table 8. Soundness sweeps and the bug-planting check of the sweep

Check	Programs	Decided	UNKNOWN	Wrong
Sweep A: affine loops (Stage 1)	2137	2137	0	**0**
Sweep B: modular loops (Stage 2)	2400	2400	0	**0**
Planted bug: threshold for 'guard false on entry' off by one	2137	2137	0	103
Planted bug: wrong sign in the ranking-function test	2137	2137	0	632
Planted bug: != rule ignores the direction of travel	2137	2137	0	60

7.3 Does the sweep actually catch bugs?

A test that never fails proves little, so we also checked the sweep itself. We planted three bugs in the static analyzer, one at a time, and re-ran Sweep A (last three rows of Table 8). The sweep reported 103, 632 and 60 wrong verdicts respectively, so it detects faulty rules, and the clean analyzer's zero is meaningful. This check is part of the test suite.

7.4 Test suite

The suite has 39 tests and passes in about 51 seconds (Fig. 13). It covers the language front end and k-bit wrapping, the static rules including the cases that must *not* be claimed (loops with a

break, non-affine updates), the cycle detector and its budget, the pipeline, the benchmark, both sweeps and the bug-planting check, the five diagonal refutations, and the three Turing-machine outcomes.

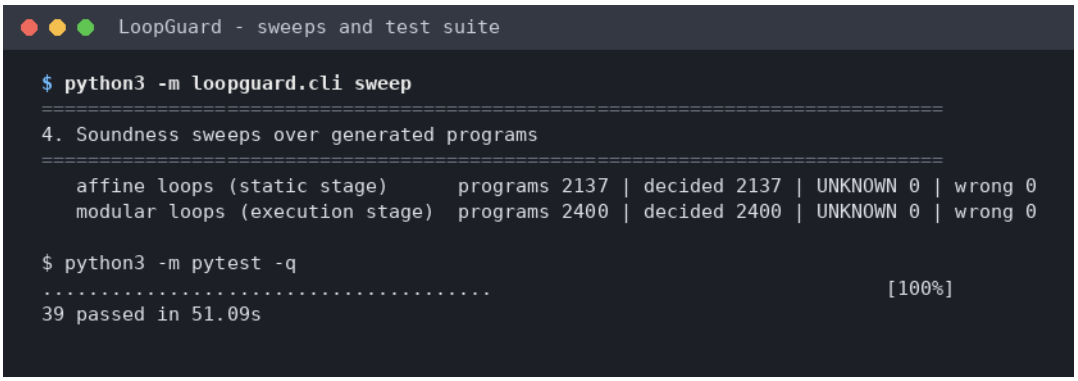


Fig. 13. The two soundness sweeps and the full test-suite run

7.5 Threats to validity

- The benchmark was written and hand-checked by us, and Sweep A lies inside the class Stage 1 is designed for; they show soundness and the value of each stage, not performance on arbitrary code.
- The oracle for Sweep A relies on the small parameter range stated above; it is independent of LoopGuard but not a proof.
- Timing and the 51-second test run are from one machine and are indicative only.

Chapter 8 Practical Applications and Limitations

8.1 Practical implications

- **No tool can promise to find every infinite loop.** Industrial termination provers such as Terminator [5, 11] work the way LoopGuard does: they are sound on a class of programs and answer ‘don't know’ outside it. A tool that never answers ‘don't know’ must sometimes be wrong.
- **Timeouts are policies, not decisions.** Watchdog timers and request timeouts are necessary in production, but our benchmark shows a timeout calling a correct, slow program an infinite loop. A timeout should be read as ‘too slow for our service level’, not as ‘proven to hang’.
- **Recognisable but not co-recognisable has a practical meaning.** Monitoring can always confirm that a job finished, but it can never confirm that a job is stuck. This is why systems use heartbeats and progress counters, which turn the question into ‘has anything changed recently?’, a question that can be answered.
- **Bounded systems are decidable in principle.** Embedded controllers with fixed-width registers are finite automata, so exact loop checking is possible, which is the idea behind model checking [10]. The limit there is state-space size, not undecidability.
- **Write analysable code.** Loops with an obvious decreasing counter, bounded retries and explicit break conditions are exactly the ones a tool can prove; our static stage proved them instantly and needed the execution stage for arithmetic loops such as gcd and Collatz. Termination of even simple linear loops is a studied problem in its own right [12, 15].
- **Typical uses.** Warnings in a code-review or build step for loops whose counter moves away from the bound, regression tests for ‘this routine must terminate on these inputs’, and teaching, because every verdict comes with a reason.

8.2 Limitations

- LoopLang has no functions, arrays, recursion, floating point or input during the run, and programs must be deterministic; real concurrent, interactive or non-deterministic software lies outside the model.
- Stage 1 understands only affine guards whose variables change by constants. It cannot prove loops such as doubling, Fibonacci growth or Euclid's algorithm, which need richer ranking functions.
- Stage 2 finds only loops that revisit a configuration, and it gives up after a fixed budget. A program that needs more steps than the budget to halt or to repeat is reported UNKNOWN.
- The k-bit model is exact only when the state space fits in the cap; with many variables it is also reported UNKNOWN.
- The evaluation is small and partly self-made (Section 7.5); we make no claim about performance on production code.

8.3 Future work

Natural next steps are lexicographic and piecewise ranking functions for loops such as gcd, synthesising linear ranking functions by constraint solving as in [6], extending non-termination

proofs beyond simple recurrence [7], and applying the same three-valued design to real Python code through its abstract syntax tree.

Chapter 9 Individual Contribution Statements

Rishika Vamsharaj (RA2411026050069), theory and formal modelling. I wrote the formal model of the problem (Section 2.1), the undecidability and recognisability argument and the discussion of Rice's theorem (Chapter 3), the executable diagonal argument with its five candidate deciders, and the Turing-machine simulator with its three examples. I also drafted Chapters 1 to 3.

Sreelakshmi A (RA2411026050058), design, implementation and validation. I designed LoopLang and its register-machine compiler and wrote the static analyzer with its ranking-function, recurrent-set and constant-step rules, the cycle-detection stage and the k-bit finite-state model. I built the 18-program benchmark and worked out the true behaviour of each program by hand, wrote the test suite with both soundness sweeps and the bug-planting check, and drafted Chapters 4 to 8.

Both of us prepared the diagrams and the presentation, and both reviewed the benchmark table, the results and the final report together.

Chapter 10 Source Code and Reproducibility

The complete code, tests, logs and diagram sources are supplied with this report as the package *loopguard_task4_code*. Public repository link: **[to be added by the team after upload]**. Only Python 3.9 or later is needed; pytest is needed for the tests, and Graphviz and Matplotlib only to redraw the figures. Every result in this report can be reproduced from the package folder with:

```
pip install pytest
python3 -m pytest -q                                  # 39 tests
python3 -m loopguard.cli all                          # diagonal demo, benchmark, sweeps
python3 -m loopguard.cli explain gcd_zero_bug         # one program, every stage
```

The folder *logs/* contains the captured output of each command, from which the terminal figures of this report were rendered; the figures are renderings of those logs, not photographs of a screen. The tables of Chapter 7 are produced by the same code that the tests exercise, so the report and the package cannot disagree.

Chapter 11 Conclusion

The question ‘will this program hang?’ has no general algorithmic answer. We stated it as membership in LOOP, the complement of HALT, proved that HALT is undecidable and that LOOP is not even recognisable, and watched five plausible deciders fail on their own diagonal programs. We then built a detector that is honest about that limit. LoopGuard decided 15 of 18 benchmark programs with justifications, left 3 as UNKNOWN, and gave no wrong answer on the benchmark or on 4 537 generated programs, while a timeout-only watchdog wrongly called a slow, correct program an infinite loop. The k-bit model showed the other side of the theory: with bounded memory the same question becomes decidable.

Limitations and future work. LoopGuard's static stage handles only loops whose guard variables change by constants, and LoopLang has no functions or arrays. Natural next steps are lexicographic ranking functions for loops such as gcd, and running the same ideas on real Python code through its AST.

References

[1] M. Sipser, *Introduction to the Theory of Computation*, 3rd ed. Boston, MA, USA: Cengage Learning, 2013.

[2] J. E. Hopcroft, R. Motwani and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Boston, MA, USA: Pearson, 2007.

[3] A. M. Turing, “On computable numbers, with an application to the Entscheidungsproblem,” *Proc. London Mathematical Society*, ser. 2, vol. 42, no. 1, pp. 230–265, 1937.

[4] H. G. Rice, “Classes of recursively enumerable sets and their decision problems,” *Trans. American Mathematical Society*, vol. 74, no. 2, pp. 358–366, 1953.

[5] B. Cook, A. Podelski and A. Rybalchenko, “Termination proofs for systems code,” in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2006, pp. 415–426.

[6] A. Podelski and A. Rybalchenko, “A complete method for the synthesis of linear ranking functions,” in *Verification, Model Checking, and Abstract Interpretation (VMCAI)*, LNCS 2937, Springer, 2004, pp. 239–251.

[7] A. Gupta, T. A. Henzinger, R. Majumdar, A. Rybalchenko and R.-G. Xu, “Proving non-termination,” in *Proc. ACM SIGPLAN-SIGACT Symp. Principles of Programming Languages (POPL)*, 2008, pp. 147–158.

[8] R. P. Brent, “An improved Monte Carlo factorization algorithm,” *BIT Numerical Mathematics*, vol. 20, no. 2, pp. 176–184, 1980.

[9] R. W. Floyd, “Assigning meanings to programs,” in *Mathematical Aspects of Computer Science*, Proc. Symposia in Applied Mathematics, vol. 19, American Mathematical Society, 1967, pp. 19–32.

[10] E. M. Clarke, O. Grumberg and D. A. Peled, *Model Checking*. Cambridge, MA, USA: MIT Press, 1999.

[11] B. Cook, A. Podelski and A. Rybalchenko, “Proving program termination,” *Communications of the ACM*, vol. 54, no. 5, pp. 88–98, 2011.

[12] A. Tiwari, “Termination of linear programs,” in *Computer Aided Verification (CAV)*, LNCS 3114, Springer, 2004, pp. 70–82.

[13] J. C. Lagarias, “The 3x + 1 problem and its generalizations,” *American Mathematical Monthly*, vol. 92, no. 1, pp. 3–23, 1985.

[14] Python Software Foundation, “ast — Abstract Syntax Trees,” *The Python Standard Library*, Python 3 documentation. [Online]. Available: https://docs.python.org/3/library/ast.html

[15] M. Braverman, “Termination of integer linear programs,” in *Computer Aided Verification (CAV)*, LNCS 4144, Springer, 2006, pp. 372–385.